Яковець Арина, ФФ-41

ЛАБОРАТОРНА РОБОТА №7. Наближене розв'язання звичайних диференціальних рівнянь

Варіант 15

Мета: Закріплення знань із застосування методів чисельного розв'язання звичайних диференціальних рівнянь (ДР).

1.

Варіант	*ДР або система ДР*	*Початкові умови*
Визначити розв'язок рівняння, застосовуючи метод невизначених коефіцієнтів		
15	$y'' - xy' - 2y = e^{-x^2}$	$y(0) = 1\ ,\ y'(0) = \frac{1}{2}$

Код реалізації:

```
import sympy as sp

x = sp.Symbol('x')

# кількість коеф
N = 8

# коеф c0, c1, ...
c = sp.symbols(f'c0:{N}')

# ряд
y = sum(c[n] * x**n for n in range(N))

# похідні
y1 = sp.diff(y, x)
y2 = sp.diff(y1, x)


r = sp.series(sp.exp(-x**2), x, 0, N).removeO()
left = y2 - x*y1 - 2*y

expr = sp.expand(left - r)

equations = []

# початкові умови: c0 = 1, c1 = 1/2
equations.append(sp.Eq(c[0], 1))
equations.append(sp.Eq(c[1], sp.Rational(1, 2)))

for k in range(N - 2):
    coef = sp.expand(expr).coeff(x, k)
    equations.append(sp.Eq(coef, 0))

solution = sp.solve(equations, c, dict=True)[0]

print("Коефіцієнти:")
for n in range(N):
    print(f"c{n} =", solution[c[n]])

y_series = y.subs(solution)

print()
print("Наближений розв'язок:")
print("y(x) =", y_series)
```

Результат:

```
Коефіцієнти:
c0 = 1
c1 = 1/2
c2 = 3/2
c3 = 1/4
c4 = 5/12
c5 = 1/16
c6 = 1/10
c7 = 1/96

Наближений розв'язок:
y(x) = x**7/96 + x**6/10 + x**5/16 + 5*x**4/12 + x**3/4 + 3*x**2/2 + x/2 + 1
```

2.

Варіант	*ДР або система ДР*	*Початкові умови*
Визначити три послідовних наближення розв'язку ДР (системи ДР), вважаючи $y_0(x) \equiv y_0$, $z_0(x) \equiv z_0$		
15	$xy' = 2x - y$	$-y(1) = 2$

Код реалізації:

Метод послідовних наближень

```
x, t = sp.symbols('x t', positive=True)

# початкові умови
x0 = 1
y_initial = 2

def f(t, y):
    return 2 - y / t

# початкове наближення
y0 = sp.Integer(2)

# 3 наближення
y1 = y_initial + sp.integrate(f(t, y0), (t, x0, x))
y2 = y_initial + sp.integrate(f(t, y1.subs(x, t)), (t, x0, x))
y3 = y_initial + sp.integrate(f(t, y2.subs(x, t)), (t, x0, x))

print("y0(x) =", y0)
print("y1(x) =", sp.simplify(y1))
print("y2(x) =", sp.simplify(y2))
print("y3(x) =", sp.simplify(y3))
```

Результат:

```
y0(x) = 2
y1(x) = 2*x - 2*log(x)
y2(x) = log(x)**2 + 2
y3(x) = 2*x - log(x)**3/3 - 2*log(x)
```

3. Застосовуючи метод Ейлера, чисельно розв'язати ДР (систему ДР) з даними початковими умовами на відрізку $[a;b]$ з кроком $h=0.1$ при вказаних значеннях параметру

Варіант	*ДР або система ДР*	*Початкові умови*
15	$y' = 1+0.6\,y\sin x-\beta y^2,$ $\beta = 1+0.25n,\ n=0,\ldots,4$	$y(0)=0, a=0, b=1$

Код реалізації:

```
import math

# параметри
a = 0
b = 1
h = 0.1

betas = [1 + 0.25 * n for n in range(5)]

for beta in betas:

    print()
    print(f"beta = {beta}")

    x = 0.0
    y = 0.0

    print(f"{'i':>2} {'x':>6} {'y':>12}")

    i = 0
    print(f"{i:2d} {x:6.1f} {y:12.6f}")

    while x < b:

        f = 1 + 0.6 * y * math.sin(x) - beta * y**2

        y = y + h * f

        x = round(x + h, 10)

        i += 1

        print(f"{i:2d} {x:6.1f} {y:12.6f}")
```

Результат:

```
beta = 1.0
 i      x           y
 0    0.0      0.000000
 1    0.1      0.100000
 2    0.2      0.199599
 3    0.3      0.297994
 4    0.4      0.394398
 5    0.5      0.488058
 6    0.6      0.578277
 7    0.7      0.664428
 8    0.8      0.745964
 9    0.9      0.822425
10    1.0      0.893440

beta = 1.25
 i      x           y
 0    0.0      0.000000
 1    0.1      0.100000
 2    0.2      0.199349
 3    0.3      0.296758
 4    0.4      0.391011
 5    0.5      0.481036
 6    0.6      0.565949
 7    0.7      0.645085
 8    0.8      0.718003
 9    0.9      0.784466
10    1.0      0.844412

beta = 1.5
 i      x           y
 0    0.0      0.000000
 1    0.1      0.100000
 2    0.2      0.199099
 3    0.3      0.295526
 4    0.4      0.387666
 5    0.5      0.474181
 6    0.6      0.554094
 7    0.7      0.626813
 8    0.8      0.692107
 9    0.9      0.750044
10    1.0      0.800911

beta = 1.75
 i      x           y
 0    0.0      0.000000
 1    0.1      0.100000
 2    0.2      0.198849
 3    0.3      0.294300
 4    0.4      0.384361
 5    0.5      0.467488
 6    0.6      0.542690
 7    0.7      0.609536
 8    0.8      0.668078
 9    0.9      0.718726
10    1.0      0.762106

beta = 2.0
 i      x           y
 0    0.0      0.000000
 1    0.1      0.100000
 2    0.2      0.198599
 3    0.3      0.293078
 4    0.4      0.381096
 5    0.5      0.460953
 6    0.6      0.531717
 7    0.7      0.593186
 8    0.8      0.645741
 9    0.9      0.690138
10    1.0      0.727316
```

5. Застосовуючи метод Ейлера з подальшою ітераційної обробкою, визначити з точністю до 10^{-4} розв'язок ДР при заданих початкових умовах на відрізку $[0;1]$ з кроком $h = 0.1$.

Варіант	*ДР або система ДР*	*Початкові умови*
	$y' = -y^2 + \frac{\beta x}{1+x^2}$, $\beta = 2.5 + \frac{\alpha}{40}$, $\alpha = 34,\ldots,37$	$y(0) = -0.4122$
15	$y' = -y^2 + \frac{\beta x}{1+x^2}$, $\beta = 2.5 + \frac{\alpha}{40}$, $\alpha = 15,\ldots,20$	

Код реалізації:

```python
a = 0
b = 1
h = 0.1
y0 = -0.4122

def f(x, y, beta):
    return -y**2 + beta * x / (1 + x**2)

for alpha in range(15, 21):
    beta = 2.5 + alpha / 40

    print()
    print(f"alpha = {alpha}, beta = {beta:.4f}")
    print(f"{'i':>2} {'x_i':>6} {'y_i':>10}")

    x = a
    y = y0

    print(f"{0:2d} {x:6.1f} {y:10.4f}")

    i = 0

    while x < b:
        x_next = round(x + h, 10)

        # грубе наближення 7.35
        y_iter = y + h * f(x, y, beta)

        # ітераційна обробка 7.36
        while True:
            y_new = y + h / 2 * (
                f(x, y, beta) + f(x_next, y_iter, beta)
            )

            if round(y_new, 4) == round(y_iter, 4):
                break

            y_iter = y_new

        y = y_new
        x = x_next
        i += 1

        print(f"{i:2d} {x:6.1f} {y:10.4f}")
```

Результат:

```
alpha = 15, beta = 2.8750
 i     x_i         y_i
 0     0.0     -0.4122
 1     0.1     -0.4151
 2     0.2     -0.3894
 3     0.3     -0.3354
 4     0.4     -0.2551
 5     0.5     -0.1525
 6     0.6     -0.0328
 7     0.7      0.0976
 8     0.8      0.2321
 9     0.9      0.3644
10     1.0      0.4891

alpha = 16, beta = 2.9000
 i     x_i         y_i
 0     0.0     -0.4122
 1     0.1     -0.4149
 2     0.2     -0.3889
 3     0.3     -0.3342
 4     0.4     -0.2531
 5     0.5     -0.1494
 6     0.6     -0.0286
 7     0.7      0.1029
 8     0.8      0.2384
 9     0.9      0.3715
10     1.0      0.4968

alpha = 17, beta = 2.9250
 i     x_i         y_i
 0     0.0     -0.4122
 1     0.1     -0.4148
 2     0.2     -0.3884
 3     0.3     -0.3331
 4     0.4     -0.2511
 5     0.5     -0.1464
 6     0.6     -0.0245
 7     0.7      0.1082
 8     0.8      0.2446
 9     0.9      0.3785
10     1.0      0.5045

alpha = 18, beta = 2.9500
 i     x_i         y_i
 0     0.0     -0.4122
 1     0.1     -0.4147
 2     0.2     -0.3878
 3     0.3     -0.3319
 4     0.4     -0.2491
 5     0.5     -0.1433
 6     0.6     -0.0203
 7     0.7      0.1134
 8     0.8      0.2509
 9     0.9      0.3856
10     1.0      0.5121

alpha = 19, beta = 2.9750
 i     x_i         y_i
 0     0.0     -0.4122
 1     0.1     -0.4146
 2     0.2     -0.3873
 3     0.3     -0.3307
 4     0.4     -0.2470
 5     0.5     -0.1403
 6     0.6     -0.0161
 7     0.7      0.1186
 8     0.8      0.2571
 9     0.9      0.3926
10     1.0      0.5197

alpha = 20, beta = 3.0000
 i     x_i         y_i
 0     0.0     -0.4122
 1     0.1     -0.4144
 2     0.2     -0.3868
 3     0.3     -0.3296
 4     0.4     -0.2450
 5     0.5     -0.1372
 6     0.6     -0.0120
 7     0.7      0.1239
 8     0.8      0.2633
 9     0.9      0.3996
10     1.0      0.5273
```

6. Методом Рунге-Кутта 4-го порядку визначити розв'язок ДР (системи ДР) із заданими кроком та початковими умовами на вказаному відрізку

Варіант	*ДР або система ДР, відрізок, крок*	*Початкові умови*
15	$x' = e^{-3t}\left(x^2 + b\right), b = 1.0 + 0.4k, k = 0,\ldots,5, [0;0.3], h = 0.1$	$x(0) = 0$

Код реалізації:

```python
h = 0.1
a = 0
T = 0.3

def f(t, x, b):
    return math.exp(-3*t) * (x**2 + b)

for k in range(6):
    b = 1 + 0.4 * k

    print()
    print(f"k = {k}, b = {b:.1f}")
    print(f"{'i':>2} {'t_i':>6} {'x_i':>10} {'K1':>10} {'K2':>10} {'K3':>10} {'K4':>10} {'dx':>10}")

    t = a
    x = 0.0
    i = 0

    while t < T:
        K1 = h * f(t, x, b)
        K2 = h * f(t + h/2, x + K1/2, b)
        K3 = h * f(t + h/2, x + K2/2, b)
        K4 = h * f(t + h, x + K3, b)

        dx = (K1 + 2*K2 + 2*K3 + K4) / 6

        print(f"{i:2d} {t:6.1f} {x:10.5f} {K1:10.5f} {K2:10.5f} {K3:10.5f} {K4:10.5f} {dx:10.5f}")

        x = x + dx
        t = round(t + h, 10)
        i += 1

    print(f"{i:2d} {t:6.1f} {x:10.5f}")
```

Результат:

```
k = 0, b = 1.0
 i    t_i        x_i         K1         K2         K3         K4         dx
 0    0.0    0.00000    0.10000    0.08629    0.08623    0.07463    0.08661
 1    0.1    0.08661    0.07464    0.06474    0.06467    0.05614    0.06493
 2    0.2    0.15154    0.05614    0.04876    0.04870    0.04229    0.04889
 3    0.3    0.20043

k = 1, b = 1.4
 i    t_i        x_i         K1         K2         K3         K4         dx
 0    0.0    0.00000    0.14000    0.12092    0.12081    0.10480    0.12138
 1    0.1    0.12138    0.10481    0.09119    0.09105    0.07931    0.09143
 2    0.2    0.21281    0.07932    0.06914    0.06902    0.06015    0.06930
 3    0.3    0.28211

k = 2, b = 1.8
 i    t_i        x_i         K1         K2         K3         K4         dx
 0    0.0    0.00000    0.18000    0.15562    0.15545    0.13514    0.15621
 1    0.1    0.15621    0.13516    0.11797    0.11773    0.10290    0.11824
 2    0.2    0.27445    0.10292    0.09004    0.08985    0.07858    0.09021
 3    0.3    0.36467

k = 3, b = 2.2
 i    t_i        x_i         K1         K2         K3         K4         dx
 0    0.0    0.00000    0.22000    0.19040    0.19014    0.16566    0.19112
 1    0.1    0.19112    0.16569    0.14506    0.14471    0.12693    0.14536
 2    0.2    0.33648    0.12695    0.11148    0.11119    0.09759    0.11165
 3    0.3    0.44813

k = 4, b = 2.6
 i    t_i        x_i         K1         K2         K3         K4         dx
 0    0.0    0.00000    0.26000    0.22524    0.22488    0.19636    0.22610
 1    0.1    0.22610    0.19640    0.17249    0.17200    0.15139    0.17280
 2    0.2    0.39889    0.15142    0.13346    0.13306    0.11721    0.13361
 3    0.3    0.53250

k = 5, b = 3.0
 i    t_i        x_i         K1         K2         K3         K4         dx
 0    0.0    0.00000    0.30000    0.26015    0.25967    0.22724    0.26115
 1    0.1    0.26115    0.22730    0.20025    0.19961    0.17629    0.20055
 2    0.2    0.46170    0.17634    0.15599    0.15547    0.13746    0.15612
 3    0.3    0.61782
```

7.

Варіант	*ДР або система ДР*
	Методом Адамса продовжити на два кроки таблицю значень розв'язків наступних рівнянь з початковими умовами $x(0)=0$. Взяти крок обчислень $h=0.1$. «Початковий відрізок» визначити методом Рунге-Кутта
15	$x'=e^{-at}\left(x^2+1\right),\ a=1.0+0.4k,\ k=0,\ldots,5$

Код реалізації:

```
h = 0.1

def f(t, x, a):
    return math.exp(-a * t) * (x**2 + 1)

def rk4_step(t, x, a):
    K1 = h * f(t, x, a)
    K2 = h * f(t + h/2, x + K1/2, a)
    K3 = h * f(t + h/2, x + K2/2, a)
    K4 = h * f(t + h, x + K3, a)

    dx = (K1 + 2*K2 + 2*K3 + K4) / 6
    return x + dx

for k in range(6):
    a = 1 + 0.4 * k

    print()
    print(f"k = {k}, a = {a:.1f}")
    print(f"{'i':>2} {'t_i':>6} {'x_i':>10} {'x*':>10}")

    t_values = [0.0]
    x_values = [0.0]

    # початковий відрізок методом Рунге-Кутта: x1, x2, x3
    t = 0.0
    x = 0.0

    for i in range(3):
        x = rk4_step(t, x, a)
        t = round(t + h, 10)

        t_values.append(t)
        x_values.append(x)

    # два кроки методом Адамса: x4, x5
    for i in range(3, 5):
        f_i   = f(t_values[i],   x_values[i],   a)
        f_i1  = f(t_values[i-1], x_values[i-1], a)
        f_i2  = f(t_values[i-2], x_values[i-2], a)
        f_i3  = f(t_values[i-3], x_values[i-3], a)

        # прогноз за формулою Адамса-Бешфорта
        x_pred = x_values[i] + h / 24 * (
            55*f_i - 59*f_i1 + 37*f_i2 - 9*f_i3
        )
```

```
        t_next = round(t_values[i] + h, 10)

        f_pred = f(t_next, x_pred, a)

        # уточнення за формулою Адамса-Мултона
        x_corr = x_values[i] + h / 24 * (
            9*f_pred + 19*f_i - 5*f_i1 + f_i2
        )

        t_values.append(t_next)
        x_values.append(x_corr)

    for i in range(len(t_values)):
        fp = f(t_values[i], x_values[i], a)
        print(f"{i:2d} {t_values[i]:6.1f} {x_values[i]:10.5f} {fp:10.5f}")
```

Результат:

```
k = 0, a = 1.0
 i    t_i        x_i         x*
 0    0.0    0.00000    1.00000
 1    0.1    0.09545    0.91308
 2    0.2    0.18328    0.84623
 3    0.3    0.26515    0.79290
 4    0.4    0.34217    0.74880
 5    0.5    0.41512    0.71105

k = 1, a = 1.4
 i    t_i        x_i         x*
 0    0.0    0.00000    1.00000
 1    0.1    0.09359    0.87697
 2    0.2    0.17623    0.77926
 3    0.3    0.24999    0.69811
 4    0.4    0.31623    0.62833
 5    0.5    0.37593    0.56677

k = 2, a = 1.8
 i    t_i        x_i         x*
 0    0.0    0.00000    1.00000
 1    0.1    0.09177    0.84231
 2    0.2    0.16956    0.71773
 3    0.3    0.23605    0.61522
 4    0.4    0.29313    0.52858
 5    0.5    0.34218    0.45417

k = 3, a = 2.2
 i    t_i        x_i         x*
 0    0.0    0.00000    1.00000
 1    0.1    0.09001    0.80902
 2    0.2    0.16323    0.66120
 3    0.3    0.22321    0.54260
 4    0.4    0.27247    0.44558
 5    0.5    0.31290    0.36546

k = 4, a = 2.6
 i    t_i        x_i         x*
 0    0.0    0.00000    1.00000
 1    0.1    0.08829    0.77706
 2    0.2    0.15723    0.60922
 3    0.3    0.21137    0.47889
 4    0.4    0.25394    0.37625
 5    0.5    0.28735    0.29503

k = 5, a = 3.0
 i    t_i        x_i         x*
 0    0.0    0.00000    1.00000
 1    0.1    0.08661    0.74638
 2    0.2    0.15154    0.56142
 3    0.3    0.20043    0.42290
 4    0.4    0.23726    0.31815
 5    0.5    0.26493    0.23879
```

8. Методом Мілна з точністю до 10^{-4} на відрізку $[a;b]$ визначити розв'язки наступних рівнянь при заданих початкових умовах. «Початковий відрізок» визначити одним з розглянутих у лекції 7 методів

Варіант	*ДР*	*Початкові умови*
15	$y' = -\frac{y}{x} - y^2 \ln x$	$y(1) = 1,\ a = 1,\ b = 2$

Код реалізації:

```
h = 0.1
a = 1
b = 2

def f(x, y):
    return -y / x - y**2 * math.log(x)

def rk4_step(x, y):
    K1 = h * f(x, y)
    K2 = h * f(x + h/2, y + K1/2)
    K3 = h * f(x + h/2, y + K2/2)
    K4 = h * f(x + h, y + K3)

    return y + (K1 + 2*K2 + 2*K3 + K4) / 6

x_values = [1.0]
y_values = [1.0]

# початковий відрізок: y1, y2, y3 методом Рунге-Кутта
x = 1.0
y = 1.0

for i in range(3):
    y = rk4_step(x, y)
    x = round(x + h, 10)

    x_values.append(x)
    y_values.append(y)

# продовження методом Мілна
while x_values[-1] < b:
    i = len(x_values)

    f_i1 = f(x_values[i-1], y_values[i-1])
    f_i2 = f(x_values[i-2], y_values[i-2])
    f_i3 = f(x_values[i-3], y_values[i-3])

    # прогноз за першою формулою Мілна
    y_pred = y_values[i-4] + 4*h/3 * (2*f_i1 - f_i2 + 2*f_i3)

    x_next = round(x_values[-1] + h, 10)

    f_pred = f(x_next, y_pred)
```

```
    # корекція за другою формулою Мілна
    y_corr = y_values[i-2] + h/3 * (f_i2 + 4*f_i1 + f_pred)

    eps = abs(y_corr - y_pred) / 29

    x_values.append(x_next)
    y_values.append(y_corr)

print(f"{'i':>2} {'x_i':>6} {'y_i':>10}")

for i in range(len(x_values)):
    print(f"{i:2d} {x_values[i]:6.1f} {y_values[i]:10.4f}")
```

Результат:

i	x_i	y_i
0	1.0	1.0000
1	1.1	0.9050
2	1.2	0.8197
3	1.3	0.7436
4	1.4	0.6760
5	1.5	0.6160
6	1.6	0.5628
7	1.7	0.5156
8	1.8	0.4737
9	1.9	0.4364
10	2.0	0.4032